Introduction to NGINX

A Professional Web Server for Real Applications

- We've used PHP's built-in server (`php -S`) for learning and testing.
- It's convenient but limited and not for production.
- Real apps use proper servers like **NGINX** (fast, efficient) or **Apache** (widely used).
- We use **NGINX** for ASE 230; Simpler, faster, and better for learning modern PHP deployment.

What is NGINX?

NGINX (pronounced "Engine-X") is a powerful, high-performance web server used by:

- **Netflix** - Video streaming
- **Airbnb** - Travel platform
- **GitHub** - Code hosting
- **Slack** - Team communication
- **Dropbox** - File storage

Key Features

- **High Performance**: Handles thousands of connections
- **Reverse Proxy**: Routes requests to backend applications
- **Static Files**: Serves images, CSS, JS efficiently
- **Load Balancing**: Distributes traffic across servers

NGINX vs PHP Built-in Server

Feature	PHP -S	NGINX
Purpose	Development only	Production ready
Performance	Limited	High performance
Concurrent Users	Few	Thousands
Static Files	Basic	Optimized
Configuration	None	Highly configurable
SSL/HTTPS	Not supported	Full support
Real World Usage	Never	Everywhere

Goal: Install NGINX and learn professional web server setup!

Installation NGINX

1. **macOS** - Using Homebrew (easiest method)
2. **Linux** - Using package managers (apt/yum)
3. **Verification** - Confirming installation works

Prerequisites

- Administrator/sudo access
- Basic command line knowledge
- No existing web servers on port 80 (stop Apache if running)

macOS Installation

Step 1: Install Homebrew (if not installed)

Open **Terminal** and run:

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

Step 2: Install NGINX

```
brew install nginx
```

Step 3: Start NGINX

```
# Start NGINX
brew services start nginx

# Or start manually
nginx
```

To stop nginx

```
# Stop NGINX
brew services stop nginx

# Or stop anually
nginx -s quit
```

Step 4: Verify Installation

Open browser and go to: **http://localhost:8080**

You should see: "Welcome to nginx!"

On macOS, NGINX uses port 8080, so make sure the port is available.

Linux Installation

Step 1: Update Package List

```
sudo apt update
```

Step 2: Install NGINX

```
sudo apt install nginx
```

Step 3: Start and Enable NGINX

```
# Start NGINX service
sudo systemctl start nginx

# Enable auto-start on boot
sudo systemctl enable nginx

# Check status
sudo systemctl status nginx
```

Stop nginx

```
sudo systemctl stop nginx
```

Step 4: Verify Installation

Open browser and go to: **http://localhost**

You should see: "Welcome to nginx!"

WSL/Ubuntu uses port 80, so make sure the port is available.

Verification Steps

1. Check if NGINX is Running

```
ps aux | grep nginx
```

2. Test Web Server

Open browser and navigate to:

- **WSL2/Ubuntu**: http://localhost
- **macOS**: http://localhost:8080

3. Check NGINX Version

```
nginx -v
```

You should see output like: `nginx version: nginx/1.18.0`

Basic NGINX Commands

Starting and Stopping NGINX

macOS:

```
# Start
brew services start nginx
# or: nginx

# Stop
brew services stop nginx
# or: nginx -s stop

# Reload
nginx -s reload
```

Linux:

```
# Start
sudo systemctl start nginx

# Stop
sudo systemctl stop nginx

# Restart
sudo systemctl restart nginx

# Reload configuration
sudo systemctl reload nginx
# or: sudo nginx -s reload

# Check status
sudo systemctl status nginx
```

Important File Locations

Configuration Files

Use `sudo nginx` WSL2/Ubuntu.

1. Use `nginx -t` to find your nginx.conf.

HTML files (For Mac)

2. $(brew --prefix)/var/www

This is an example output from Mac.

```
> nginx -t
nginx: the configuration file /opt/homebrew/etc/nginx/nginx.conf syntax is ok
nginx: configuration file /opt/homebrew/etc/nginx/nginx.conf test is successful
> $(brew --prefix)/var/www
/opt/homebrew/var/www
```

HTML files (for WSL2/Ubunt)

2. Open `/etc/nginx/sites-available/default`
3. Find the root section: `root /var/www/html;`

macOS (Homebrew):

- Main config:
  `/usr/local/etc/nginx/nginx.conf`
- HTML files: `/usr/local/var/www/`

For Apple Silicon Mac:

- Main config:
  `/opt/homebrew/etc/nginx/nginx.conf`
- HTML files: `/opt/homebrew/var/www`

WSL2/Ubuntu

You need to check the location.

- Main config: `/etc/nginx/nginx.conf`
- HTML files: `/var/www/html`
- Site configs: `/etc/nginx/sites-available/`

Testing Your Installation

1. Create a Test HTML File

Navigate to your NGINX HTML directory and find the `index.html`.

You can also create `nginx_test.html`:

```
<!DOCTYPE html>
<html>
<head>
    <title>NGINX Test</title>
</head>
<body>
    <h1>NGINX is Working!</h1>
    <p>Congratulations! You've successfully installed NGINX.</p>
    <p>Date: <span id="date"></span></p>
    <script>
        document.getElementById('date').textContent = new Date().toLocaleString();
    </script>
</body>
```

2. Test the File

Visit: **http://localhost** (or **:8080** on macOS)

Visit: **http://localhost/nginx_test.html** (or **:8080** on macOS)

```
NGINX is Working!

Congratulations! You've successfully installed NGINX.

Date: 9/1/2027, 10:31:18 AM
```

Troubleshooting Common Issues

1. Port Already in Use

Error: `bind() to 0.0.0.0:80 failed`

Solution: Another service is using port 80

```
# Check what's using port 80
sudo ss -ltnp 'sport = :80'
sudo lsof -nP -iTCP:80 -sTCP:LISTEN


# Stop Apache if running (example)
sudo systemctl stop apache2  # Ubuntu
sudo systemctl stop httpd    # CentOS
```

For mac

```
lsof -i :8080
```

2. Permission Denied (Linux)

Error: Permission denied

Solution: Use sudo or check the firewall

```
sudo nginx
sudo systemctl start nginx

# Check firewall
sudo ufw allow 'Nginx Full'  # Ubuntu
```

3. Configuration Errors

Test configuration before starting:

```
nginx -t
```

Common fix: Check syntax in `nginx.conf`

4. Can't Access from Other Devices

Solution: Configure firewall to allow HTTP (port 80)

Key Takeaways

NGINX is widely used because:

1. **Industry Standard**: Used by major websites worldwide
2. **Performance**: Handles many more users than the PHP built-in server
3. **Production Ready**: What you'll use in real applications
4. **Professional Skills**: Essential for web developers